

Chuyên đề: Căn Bậc Hai Trên Mảng & Cấu Trúc Dữ Liệu (Square Root Decomposition - SRD)

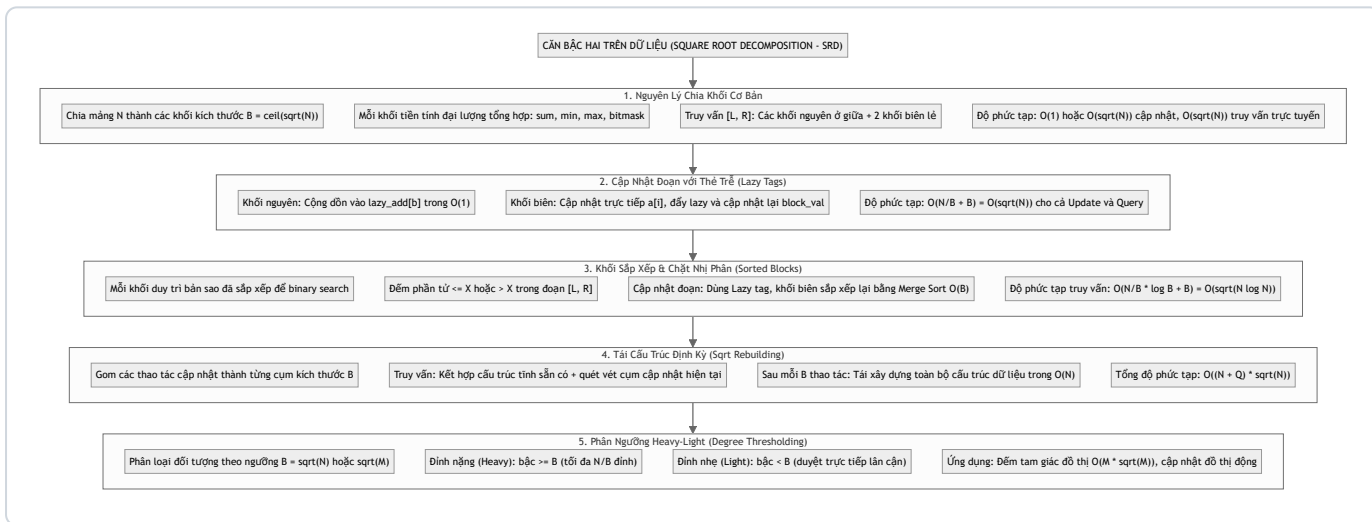
(Block Decomposition, Range Updates with Lazy, Sqrt Rebuilding, Heavy-Light Thresholding, Sorted Blocks Binary Search)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề & Bản Chất Thuật Toán Chia Căn

1.1. Bối cảnh và triết lý thiết kế

Trong xử lý dữ liệu và truy vấn đoạn trên mảng $A[0 \dots N - 1]$, chúng ta thường đứng trước sự đánh đổi (trade-off) giữa thời gian cập nhật và thời gian truy vấn:

- Mảng thô (Naive Array):** Cập nhật 1 phần tử mất $O(1)$, nhưng tính tổng đoạn $[L, R]$ mất $O(N)$.
- Mảng tiền tố (Prefix Sum Array):** Truy vấn tổng đoạn $[L, R]$ mất $O(1)$, nhưng khi cập nhật 1 phần tử thì phải tính lại toàn bộ mảng tiền tố mất $O(N)$.
- Cây phân đoạn (Segment Tree / Fenwick Tree):** Cân bằng xuất sắc với $O(\log N)$ cho cả cập nhật và truy vấn. Tuy nhiên, Segment Tree đòi hỏi phép toán phải có tính kết hợp (Associative), mã nguồn tương đối dài, và

gặp nhiều khó khăn khi bài toán yêu cầu các phép toán phức tạp (như đếm số phần tử $\leq X$, tìm kiếm nội bộ khối, hoặc các thao tác trên đồ thị).

Square Root Decomposition (Chia căn trên dữ liệu) cung cấp một giải pháp thiết kế cực kỳ linh hoạt và tự nhiên:

- Thay vì chia đôi đệ quy theo cây nhị phân độ sâu $O(\log N)$, ta chia mảng thành $O(\sqrt{N})$ khối phẳng liên tiếp, mỗi khối chứa xấp xỉ $\sqrt{N}$ phần tử.
- Mỗi thao tác cập nhật hoặc truy vấn chỉ cần xử lý tối đa $O(\sqrt{N})$ khối nguyên và $O(\sqrt{N})$ phần tử lẻ ở hai đầu mút, mang lại thời gian thực thi:

$$O(\sqrt{N})$$

- Điểm vượt trội của chia căn là tính đơn giản, dễ kết hợp với các cấu trúc dữ liệu khác (như `std::vector` đã sắp xếp bên trong từng khối), và khả năng xử lý các bài toán không thể phân rã nhị phân đơn thuần.

1.2. Phân biệt then chốt: SRD trên Mảng vs. Mo's Algorithm

Rất nhiều bạn học thường nhầm lẫn giữa **Square Root Decomposition (SRD)** và **Mo's Algorithm**. Đây là hai khái niệm hoàn toàn khác biệt:

Tiêu chí	Square Root Decomposition (SRD trên mảng)	Mo's Algorithm (Chia căn trên truy vấn)
Đối tượng phân rã	Mảng dữ liệu $A[0 \dots N - 1]$ được chia thành $\sqrt{N}$ khối tĩnh	Tập hợp Q truy vấn được chia thành các khối dựa trên đầu mút trái L
Chế độ thực thi	Trực tuyến (Online): Nhận truy vấn đến đâu trả lời ngay đến đó	Ngoại tuyến (Offline): Phải đọc toàn bộ Q truy vấn rồi sắp xếp lại
Cơ chế xử lý	Ghép kết quả của các khối trọn vẹn ở giữa với 2 khối biên lẻ	Dùng hai con trỏ $[currL, currR]$ trượt trên mảng dữ liệu gốc
Thao tác hỗ trợ	Hỗ trợ cập nhật điểm, cập nhật đoạn (kèm Lazy) trực tiếp	Rất khó hỗ trợ cập nhật (cần mở rộng sang 3D Mo $O(N^{5/3})$)
Phạm vi bài toán	Tổng đoạn, min / max, đếm phần tử $\leq X$, chia ngưỡng bậc đồ thị	Đếm số phần tử phân biệt, tần số xuất hiện, hàm không có tính cộng dồn

2. Mô Hình Chia Khối Cơ Bản: Point Update & Range Query

2.1. Phân hoạch khối và ánh xạ chỉ số

Cho mảng A gồm N phần tử đánh số từ 0 đến $N - 1$.

Ta chọn kích thước khối $B = \lceil \sqrt{N} \rceil$.

- Số lượng khối: $K = \lceil N/B \rceil \approx \sqrt{N}$.

- Phần tử tại chỉ số i sẽ thuộc về khối:

$$\text{block_idx}(i) = \lfloor i/B \rfloor$$

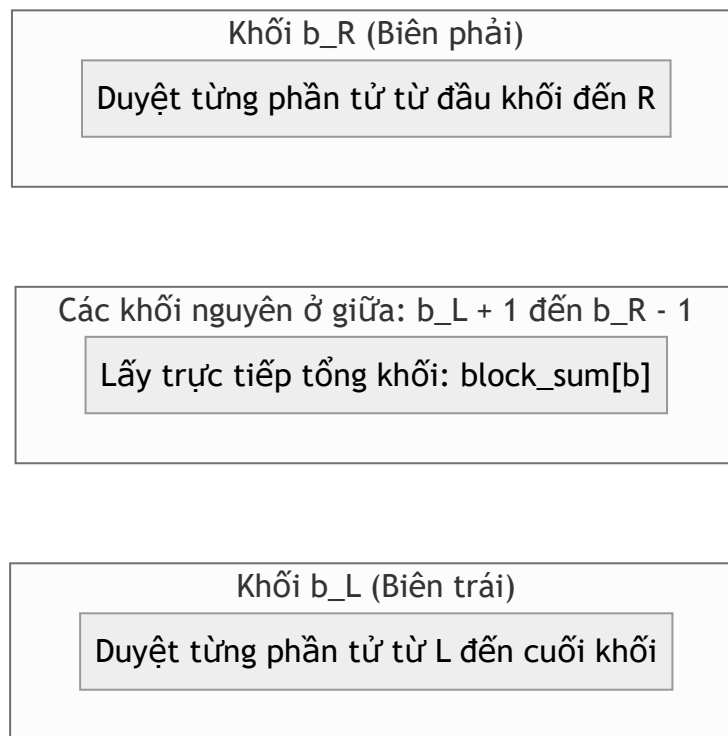
- Khối thứ b quản lý đoạn chỉ số:

$$[L_b, R_b] = [b \cdot B, \min(N - 1, (b + 1) \cdot B - 1)]$$

Chỉ số:	0	1	2		3	4	5		6	7	8				
Phần tử:	[	2	5	1	]	[	8	3	6	]	[	4	9	7	]
Khối:	Khối 0				Khối 1				Khối 2						
Tổng khối:	8				17				20						

2.2. Phân rã truy vấn đoạn $[L, R]$

Khi cần tính tổng các phần tử trong đoạn $[L, R]$, ta xác định khối chứa đầu nút trái $b_L = \lfloor L/B \rfloor$ và khối chứa đầu nút phải $b_R = \lfloor R/B \rfloor$:



Trường hợp 1: L và R thuộc cùng một khối ($b_L = b_R$)

Ta chỉ việc duyệt tuần tự từng phần tử từ L đến R :

$$\text{Thời gian: } O(R - L + 1) \leq O(B) = O(\sqrt{N})$$

Trường hợp 2: L và R thuộc hai khối khác nhau ($b_L < b_R$)

Đoạn $[L, R]$ được tách thành 3 phần rõ rệt:

1. **Phần đuôi của khối b_L** : Duyệt từng phần tử từ $i = L$ đến cuối khối b_L (tối đa B phần tử).
2. **Các khối nguyên ở giữa ($b_L + 1 \leq b \leq b_R - 1$)**: Lấy trực tiếp giá trị tiền xử lý `block_sum[b]` của từng khối trong $O(1)$. Có tối đa N/B khối nguyên.
3. **Phần đầu của khối b_R** : Duyệt từng phần tử từ đầu khối b_R đến $i = R$ (tối đa B phần tử).

Tổng thời gian cho một truy vấn:

$$T_{query} = O(B) + O\left(\frac{N}{B}\right) + O(B) = O\left(B + \frac{N}{B}\right)$$

Khi chọn $B = \lceil \sqrt{N} \rceil$, thời gian truy vấn đạt mức tối ưu:

$$O(\sqrt{N})$$

2.3. Thao tác Cập nhật điểm (Point Update)

Khi giá trị tại vị trí idx thay đổi từ $A[idx]$ thành val :

- Cập nhật giá trị mảng gốc: $A[idx] \leftarrow val$.
- Cập nhật lại thông tin của khối $b = \lfloor idx/B \rfloor$:
- Đối với bài toán tính tổng: `block_sum[b] += val - old_val` (mất $O(1)$).
- Đối với bài toán tìm min / max: Duyệt lại toàn bộ khối b để tính lại min mới (mất $O(B) = O(\sqrt{N})$).

3. Cập Nhật Đoạn với Thẻ Đánh Dấu Trễ (Range Updates with Lazy Tags)

3.1. Ý tưởng mở rộng

Tương tự như kỹ thuật Lazy Propagation trên Segment Tree, ta hoàn toàn có thể thực hiện **cập nhật đoạn (Range Add / Range Assign)** trên cấu trúc chia căn bằng cách gán một biến thẻ trễ `lazy[b]` cho mỗi khối b .

Khi cần cộng giá trị v vào tất cả phần tử trong đoạn $[L, R]$:

- **Với các khối nguyên ở giữa ($b_L + 1 \leq b \leq b_R - 1$)**:
 - Không duyệt từng phần tử của khối! Ta chỉ cập nhật thẻ trễ: `lazy[b] += v`.
 - Cập nhật tổng khối: `block_sum[b] += v * B`. Thao tác này mất $O(1)$ cho mỗi khối.
- **Với hai khối biên lẻ (b_L và b_R)**:
 - Đẩy giá trị `lazy` xuống mảng gốc của khối (nếu có): $A[i] \leftarrow A[i] + lazy[b]$ với mọi i thuộc khối, sau đó đặt `lazy[b] ← 0`.
 - Cộng trực tiếp v vào các phần tử $A[i]$ nằm trong phạm vi truy vấn.
 - Tính toán lại `block_sum` hoặc `block_min` của khối biên trong $O(B)$.

```

void range_add(int l, int r, long long val) {
    int bl = l / B, br = r / B;
    if (bl == br) {
        for (int i = l; i <= r; ++i) a[i] += val;
        recalculate_block(bl);
        return;
    }
    // Khối biên trái
    for (int i = l; i < (bl + 1) * B; ++i) a[i] += val;
    recalculate_block(bl);

    // Các khối nguyên ở giữa
    for (int b = bl + 1; b < br; ++b) {
        lazy[b] += val;
        block_sum[b] += val * B;
    }

    // Khối biên phải
    for (int i = br * B; i <= r; ++i) a[i] += val;
    recalculate_block(br);
}

```

Cả thao tác `range_add` và `range_sum` đều đạt độ phức tạp:

$$O\left(\frac{N}{B} + B\right) = O(\sqrt{N})$$

4. Khối Sắp Xếp & Chặt Nhị Phân (Sorted Blocks & Range Counting)

4.1. Bài toán đếm số phần tử $\leq X$ trong đoạn $[L, R]$

Đây là bài toán kinh điển (*SPOJ KQUERY*, *Codeforces 1234D*) mà việc cài đặt bằng Segment Tree động hoặc Persistent Segment Tree là rất phức tạp. Tuy nhiên, kỹ thuật chia căn giải quyết bài toán này cực kỳ tao nhã!

4.2. Cấu trúc dữ liệu

- Mảng dữ liệu gốc $A[0 \dots N - 1]$.
- Mỗi khối b duy trì thêm một vector `sorted_block[b]` chứa toàn bộ các phần tử của khối đó đã được sắp xếp tăng dần:

```

Khối b:      a = [ 5, 1, 8, 3 ]
sorted_block[b]: [ 1, 3, 5, 8 ]

```

4.3. Quy trình thực thi

- Truy vấn: Đếm số phần tử $\leq X$ trong đoạn $[L, R]$:**
- Với hai khối biên lẻ: Duyệt từng phần tử $A[i]$ trên mảng gốc, kiểm tra xem $A[i] + \text{lazy}[b] \leq X$ hay không. Tối đa $2B$ bước.

3. Với các khối nguyên ở giữa: Vì mảng `sorted_block[b]` đã được sắp xếp, ta có thể dùng chèn nhị phân `std::upper_bound` để đếm số phần tử $\leq X - \text{lazy}[b]$ trong thời gian $O(\log B)$!
4. Tổng thời gian truy vấn:

$$T_{\text{query}} = O\left(\frac{N}{B} \log B + B\right)$$

Nếu chọn $B \approx \sqrt{N \log N}$, độ phức tạp xấp xỉ $O(\sqrt{N \log N})$.

5. **Cập nhật: Cộng giá trị v vào đoạn $[L, R]$:**
6. Với các khối nguyên ở giữa: Chỉ cần tăng thẻ trễ `lazy[b] += v` trong $O(1)$. Thứ tự tương đối của các phần tử trong khối không đổi, nên mảng `sorted_block[b]` không cần sắp xếp lại!
7. Với hai khối biên lẻ: Cộng v vào các phần tử tương ứng trên A . Sau đó, ta tái cấu trúc lại `sorted_block[b]` bằng cách gộp hai mảng con đã sắp (Merge Sort) trong thời gian $O(B)$ thay vì dùng `std::sort` $O(B \log B)$.

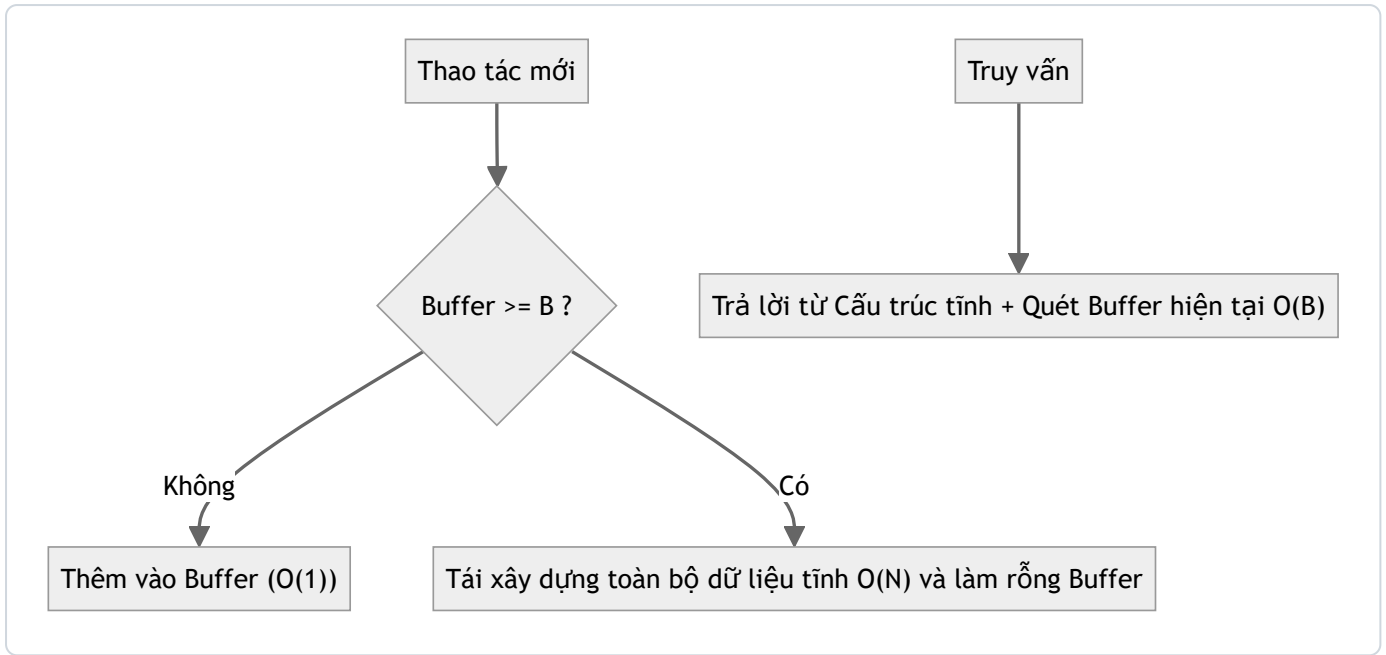
5. Kỹ Thuật Tái Cấu Trúc Định Kỳ (Square Root Rebuilding)

5.1. Thiết lý gom nhóm (Batch Processing)

Nhiều bài toán có cấu trúc dữ liệu tĩnh rất mạnh (như mảng tĩnh, đồ thị tĩnh, cây khung tĩnh cho phép trả lời truy vấn trong $O(1)$ hoặc $O(\log N)$), nhưng khi có thêm các phép chèn/xóa động thì việc duy trì trực tiếp cấu trúc đó là cực kỳ khó khăn.

Kỹ thuật Sqrt Rebuilding:

- Thay vì cập nhật cấu trúc dữ liệu sau mỗi truy vấn, ta gom các thao tác cập nhật vào một danh sách tạm thời (Buffer / Batch).
- Khi có truy vấn: Ta kết hợp kết quả từ cấu trúc tĩnh có sẵn với việc **duyệt vết toàn bộ các cập nhật trong Buffer hiện tại**.
- Khi kích thước của Buffer đạt ngưỡng $B \approx \sqrt{N}$: Ta tiến hành **xây dựng lại (Rebuild)** toàn bộ cấu trúc dữ liệu tĩnh từ đầu bằng tất cả dữ liệu tích lũy, sau đó xóa rỗng Buffer.



5.2. Phân tích độ phức tạp

- Số lần Rebuild trong suốt quá trình Q thao tác là $\frac{Q}{B}$. Mỗi lần Rebuild tốn $O(N)$ $\implies$ Tổng chi phí Rebuild là $O\left(\frac{Q \cdot N}{B}\right)$.
- Mỗi truy vấn chỉ cần duyệt qua tối đa B thao tác trong Buffer hiện tại $\implies$ Chi phí trả lời truy vấn là $O(Q \cdot B)$.
- Tổng độ phức tạp:

$$f(B) = O\left(\frac{Q \cdot N}{B} + Q \cdot B\right)$$

Chọn $B = \lceil \sqrt{N} \rceil$, ta đạt độ phức tạp toàn cục:

$$O((N + Q)\sqrt{N})$$

6. Phân Ngưỡng Heavy-Light (Degree Sqrt Thresholding)

6.1. Nguyên lý phân loại bậc trên đồ thị

Trong một đồ thị có N đỉnh và M cạnh, tổng bậc của các đỉnh là $2M$:

$$\sum_u \deg(u) = 2M$$

Ta chọn một ngưỡng phân loại $B \approx \sqrt{M}$:

1. **Đỉnh Nặng (Heavy Vertex):** Các đỉnh có $\deg(u) \geq B$.

- **Đặc điểm sống còn:** Vì tổng bậc là $2M$, nên số lượng đỉnh nặng tối đa là:

$$\text{Count}(\text{Heavy}) \leq \frac{2M}{B} \approx O(\sqrt{M})$$

2. **Đỉnh Nhẹ (Light Vertex):** Các đỉnh có $\deg(u) < B$.

- **Đặc điểm:** Số lượng đỉnh nhẹ có thể nhiều, nhưng bậc của mỗi đỉnh nhẹ luôn nhỏ hơn B . Duyệt qua toàn bộ lân cận của đỉnh nhẹ chỉ mất tối đa $O(B) = O(\sqrt{M})$.

6.2. Ứng dụng kinh điển: Đếm số tam giác trong đồ thị (Triangle Counting)

Cho đồ thị đơn vô hướng $G = (V, E)$ gồm N đỉnh và M cạnh. Cần đếm số bộ ba đỉnh (u, v, w) sao cho có đủ 3 cạnh $(u, v), (v, w), (w, u)$.

Thuật toán chia căn định hướng:

Ta định hướng lại toàn bộ các cạnh vô hướng của đồ thị thành đồ thị có hướng không chu trình (DAG) theo quy tắc:

$$u \rightarrow v \iff \deg(u) < \deg(v) \text{ hoặc } (\deg(u) == \deg(v) \text{ và } u < v)$$

Khi đó, với mọi đỉnh u , bán bậc ra $\deg^+(u)$ luôn bị chặn trên bởi $O(\sqrt{M})$:

- Nếu u là đỉnh nhẹ: $\deg^+(u) \leq \deg(u) < B = O(\sqrt{M})$.

- Nếu u là đỉnh nặng: Các đỉnh v mà $u \rightarrow v$ buộc phải có bậc lớn hơn hoặc bằng bậc của u , nghĩa là v cũng phải là đỉnh nặng. Mà số lượng đỉnh nặng tối đa là $\frac{2M}{B} = O(\sqrt{M})$.

Quy trình đếm:

1. Khởi tạo mảng đánh dấu `visited[x] = 0`.

2. Với mỗi đỉnh $u \in V$:

- Đánh dấu tất cả các đỉnh kề theo cạnh hướng ra: `visited[v] = u` với mọi $u \rightarrow v$.

- Với mỗi đỉnh v mà $u \rightarrow v$, duyệt tiếp các đỉnh w mà $v \rightarrow w$:

- Nếu `visited[w] == u`, ta phát hiện một tam giác (u, v, w) !

- Xóa đánh dấu của các đỉnh v .

```
long long count_triangles(int n, const vector<vector<int>>& out_adj) {
    vector<int> tag(n + 1, 0);
    long long ans = 0;
    for (int u = 1; u <= n; ++u) {
        for (int v : out_adj[u]) tag[v] = u;
        for (int v : out_adj[u]) {
            for (int w : out_adj[v]) {
                if (tag[w] == u) ans++;
            }
        }
    }
    return ans;
}
```

Độ phức tạp: Mỗi cạnh (u, v) chỉ duyệt qua $\deg^+(v) \leq O(\sqrt{M})$ lần cận hướng ra. Tổng thời gian thực thi:

$$O\left(\sum_{(u,v) \in E} \deg^+(v)\right) = O(M\sqrt{M})$$

Với $M = 2 \times 10^5$, $M\sqrt{M} \approx 2 \times 10^5 \times 450 \approx 9 \times 10^7$ phép tính, chạy mượt mà trong **0.15 giây!**

7. Bảng So Sánh Các Cấu Trúc Dữ Liệu Đoạn

Cấu trúc dữ liệu	Cập nhật điểm	Cập nhật đoạn (Lazy)	Truy vấn đoạn	Bộ nhớ	Độ phức tạp cài đặt	Ưu điểm vượt trội
Fenwick Tree (BIT)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$ phẳng	Cực kỳ ngắn gọn (15 dòng)	Hằng số CPU nhỏ nhất, cache tối ưu
Segment Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(4N)$	Trung bình (50-80 dòng)	Tổng quát cho mọi Monoid có tính kết hợp
Square Root Decomposition	$O(1)$	$O(\sqrt{N})$	$O(\sqrt{N})$	$O(N)$	Rất đơn giản, trực quan	Hỗ trợ Sorted Blocks, Rebuilding, Heavy-Light
Mo's Algorithm (Offline)	Không hỗ trợ	Không hỗ trợ	Khấu hao $O(\sqrt{N})$	$O(N + Q)$	Trung bình	Xử lý các bài toán không có tính kết hợp

8. Cạm Bẫy Phổ Biến & Kinh Nghiệm Thi Đấu

1. **Khối cuối cùng bị tràn biên:** Khối cuối cùng có thể có kích thước nhỏ hơn B nếu N không chia hết cho B . Luôn dùng công thức cận phải an toàn:

$$R_b = \min(N - 1, (b + 1) \cdot B - 1)$$

2. **Chọn kích thước khối cứng nhắc:** Trong bài toán đếm phần tử $\leq X$ bằng khối sắp xếp, độ phức tạp là $O(\frac{N}{B} \log B + B)$. Khi đó chọn $B = \sqrt{N \log_2 N}$ sẽ nhanh hơn đáng kể so với cố định $B = \sqrt{N}$.
3. **Quên giải phóng lazy tag khi cập nhật khối biên:** Nếu trong khối biên có các phần tử đang chịu ảnh hưởng của `lazy[b]`, bạn bắt buộc phải đẩy `lazy[b]` xuống mảng `A` của toàn bộ khối trước khi sửa đổi trực tiếp các phần tử lẻ, nếu không các phần tử không nằm trong truy vấn sẽ bị mất giá trị cộng dồn!
4. **Cache locality:** Các mảng phụ trợ của từng khối như `block_sum`, `lazy` có kích thước chỉ khoảng $\sqrt{N} \approx 450$ phần tử, hoàn toàn nằm gọn trong bộ nhớ đệm L1 Cache (32 KB), giúp tốc độ duyệt qua

các khối nguyên diễn ra với tốc độ phần cứng tối đa.

9. Tài liệu Tham khảo & Đọc thêm

1. **CP-Algorithms:**
 2. [Sqrt Decomposition](#) — Kiến trúc phân rã căn bậc hai trên mảng, cập nhật đoạn và RMQ.
 3. **VNOI Wiki:**
 4. [Phân tách căn bậc hai \(Phần 1\)](#) — Cơ sở lý thuyết, phân hoạch khối và các bài toán mẫu.
 5. [Phân tách căn bậc hai \(Phần 2\)](#) — Kỹ thuật Rebuilding và phân ngưỡng bậc Heavy-Light.
 6. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
 7. [Chapter 9: Range Queries \(Square Root Techniques\)](#) — So sánh hiệu năng giữa chia căn và Segment Tree.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.